FixMyCity

Report local issues. Track every fix.

A closed-loop civic issue-reporting platform for Ghana

Group Zero Down Time · CSCD 602 Advanced Software Engineering · University of Ghana

22425071 Nana Agyemang Duah · 22424595 Nigel Dolling · 22424245 Samuel Owusu Ampadu
22427427 Stanford Ofori · 22424693 Alexander Adade · 22425066 Hajara Yusif

Pilot: Ayawaso West Municipal Assembly (AWMA) · 3 issue categories · 4 user roles · Progressive Web App

Why reports used to vanish — and how we close the loop

The problem

- Earlier Ghanaian reporting attempts failed: reports disappeared into a void.
- No resolution workflow — nobody owned an issue after submission.
- No feedback — citizens never learned what happened, so they stopped reporting.
- No institutional buy-in from assemblies.

Our answer: the closed loop

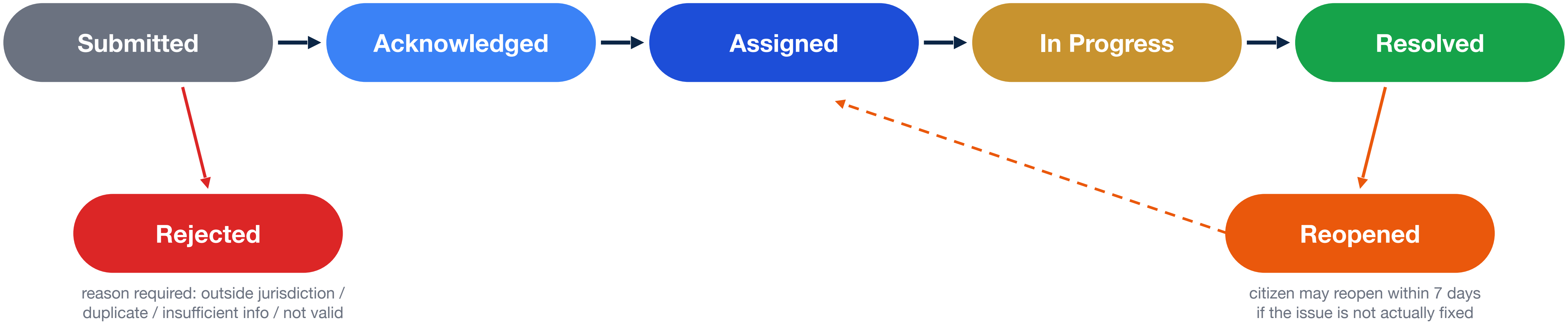
- Every report moves through a strict, server-enforced state machine.
- The citizen is notified at every single transition (in-app + email).
- Every change is timestamped and audit-logged — nothing vanishes.
- One console gives AWMA staff a single inbox, crew dispatch and analytics.

Pilot scope (deliberately locked)

Ayawaso West Municipal Assembly — ~75,303 residents, ~31 km^2, incl. UG Legon campus · Categories: Illegal Dumping, Blocked Drain, Broken Streetlight · Roles: Citizen, Reports Officer, Field Crew, Administrator · Delivered as an installable Progressive Web App

WORKFLOW

How it works — one strict state machine, closed loop



Server-side only

Front-ends never write a status. Every transition goes through the transition-report edge function, which validates legality for the current state and the caller's role.

Audited, always

Each transition writes exactly one row to an append-only audit log (DB trigger blocks updates) — surfaced in the console Audit Log.

Citizen always informed

Every transition fires an in-app realtime notification and a branded email. This is the closed loop.

Architecture and tech stack

Citizen app (PWA)

Installable, mobile-first, 3G-optimised
React + Vite + Tailwind v4
Leaflet + OpenStreetMap
Deployed on Vercel

Operations console (desktop)

Inbox · map · assignments · crews · analytics · users & roles · audit log
React + Vite + Tailwind v4

Supabase backend

- **Postgres** + PostGIS (proximity) + pgvector (image embeddings)
- **Row Level Security** on every table — zero direct client writes
- **10 Edge Functions** TypeScript/Deno — incl. the state machine
- **Auth & Storage** email auth · private photo bucket (WebP)
- **Realtime** live report + notification streams

Computer-vision API

image classification, validity gate, perceptual-hash duplicate detection

Resend

branded transactional email — status updates + auth flows

Vercel + GitHub Actions

hosting, CI, Vitest test suite

Thin clients, smart server: business rules (state machine, RBAC, jurisdiction gate, notifications) all live in edge functions — the same guarantees no matter which client calls.

The AI model — what it is, why we chose it, how we use it

One computer-vision model, built by our team, reads every report photo and answers three questions before a human ever looks at it.

WHAT it is

- **Our own CV service.** A model our team built and runs as a separate service; our app calls it over a clean API.
- **1 - Is it real?** It first checks the photo actually shows a public/environmental problem, filtering out selfies and junk.
- **2 - What kind?** YOLOv11 detects the objects in the photo and picks the category: dumping, drain or streetlight.
- **3 - Seen before?** CLIP turns the photo into a 'meaning fingerprint' and compares it (cosine similarity) to nearby recent reports; GPS distance narrows the search first.

WHY we chose it

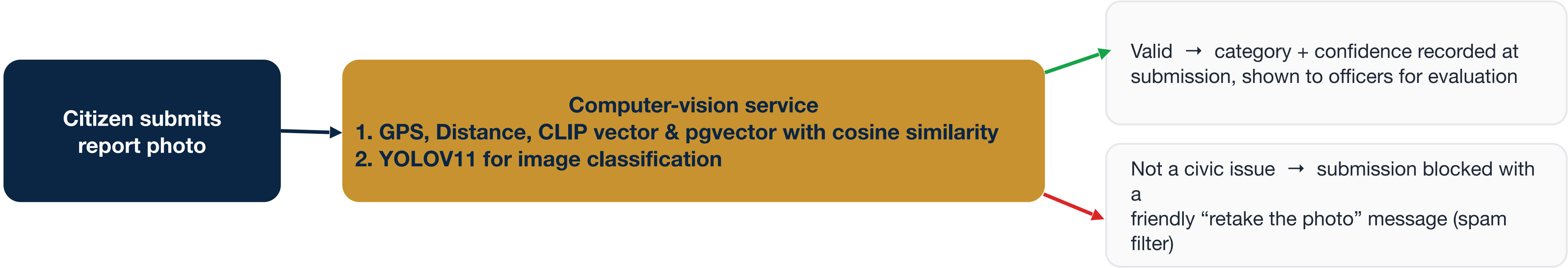
- **Runs on the server.** The citizen's cheap Android phone on 3G does zero AI work - the model runs in our backend.
- **YOLOv11 is fast + accurate.** It recognises objects from a single photo in real time - a good fit for our three categories.
- **CLIP matches meaning, not pixels.** It still spots the same rubbish pile shot from a new angle or time of day, which a pixel or filename check would miss.
- **Human-in-the-loop.** The model only suggests; a person always decides. It never auto-rejects or auto-merges a report.
- **Fail-soft.** An 8-second timeout means a slow or down model never blocks a citizen's report.

HOW we use it now

- **On submit.** The app sends the photo + GPS to the service and waits (max 8s).
- **Not a real issue.** Submission is blocked with a friendly 'retake the photo' message - a built-in spam filter.
- **Real issue.** The model's category + confidence are saved next to the citizen's own choice, so officers can compare them and we can measure accuracy. The citizen's choice always stands.
- **Looks like a duplicate.** The citizen is offered 'follow the existing report' instead of filing a new one.

AI assists, humans decide. The model only advises - our closed-loop workflow and the people running it make every real decision.

AI 1 — Auto-categorisation and photo validity gate



- **Human-in-the-loop by design.** The citizen picks the category; the AI's inferred category + confidence are stored (ai_suggested_category, ai_confidence) and shown to officers, so we can evaluate model accuracy against the citizen's choice — we never auto-classify silently.
- **Fail-soft.** The vision call has a strict timeout — if the model is slow or down, the report still goes through. AI assists reporting; it never blocks it.
- **Integrated as a service.** The classifier is a separately-built computer-vision API (a team sub-project), consumed through a thin anti-corruption client in our edge functions — clean contract, swappable model.

AI 2 — Duplicate detection, with a human making the call

LIVE IN DEMO

Image-similarity dedup at submission

The computer vision service decides;

1. Duplicate — very similar and close by(within 100m)
2. Possible duplicate — somewhat similar → human review
3. New — not similar

IN THE SCHEMA, READY

Two-stage geospatial + vector design

- **Stage 1 — PostGIS:** candidate reports = open issues within 100 m and the last 7 days of the new report.
- **Stage 2 — pgvector:** candidates ranked by cosine similarity of 512-dimension image embeddings (HNSW-indexed); top 3 returned.
- Migrated and unit-callable today (find_duplicate_candidates); the external CV service currently fulfils this role in production.

Principle: the AI flags, a human decides. We never auto-reject or auto-merge a citizen's report.

SECURITY · QUALITY · COMPLIANCE

Engineering rigor beneath the demo

Row Level Security everywhere

Citizens see their own reports, crews only their assignments, staff see all. No table accepts a direct client write — all writes go through edge functions.

Server-enforced RBAC

Five console roles (Admin → Viewer) plus field crews; every edge function re-checks the caller's permission matrix server-side.

Append-only audit trail

A database trigger makes status history immutable — even privileged code cannot rewrite it. Full traceability per report.

Jurisdiction gate

Report locations are ray-cast against the real AWMA boundary polygon (OpenStreetMap) on client and server; outside reports are rejected with guidance.

Data protection (Act 843)

Minimal PII, TLS everywhere, private photo storage — and followers of a report can never see the original reporter's identity.

Built for Ghanaian networks

Mid-range Android on 3G is the target: client-side WebP compression, small bundles, installable PWA, ≤5 s time-to-interactive budget.

Tested & traceable

Vitest unit suites (incl. 37 console tests, jurisdiction-gate tests) and every feature mapped to SRS requirements (FR-010 → FR-071).

Unique references

Human-friendly IDs like FMC-2026-0432, generated by a database trigger — quotable at the assembly, over the phone, on radio.

What you'll see in the live demo

1. **Citizen reports** — photo → AI validates it & records a category for officers → pin the location → unique reference issued
2. **AI dedup** — a similar photo triggers the duplicate offer — follow the existing report instead of re-filing
3. **Console triage** — officer acknowledges, assigns an available crew — audit log grows with every action
4. **Crew resolves** — one-tap start / resolve from the crew view
5. **Loop closes** — citizen is notified in-app + by email at every step — and can reopen within 7 days if it isn't fixed

What's next

- Web push notifications (FCM)
- Offline report queueing
- Playwright end-to-end tests
- Scale beyond AWMA to more assemblies
- Twi, Ga and Ewe localisation

Thank you — questions welcome.